LAB 1

I. Objectives

After completing this lab, students will be able to:

Knowledge

Understand the role and importance of data preprocessing in the data mining process.

Identify different types of data objects and attribute types.

Understand common preprocessing techniques, including:

1. Data cleaning
2. Data integration
3. Data reduction
4. Data transformation
5. Data discretization

Skills

- Identify common data issues such as missing data, noise, duplication, and inconsistency.
- Select and apply appropriate preprocessing methods for different types of data.
- Use Python tools to perform preprocessing steps.
- Prepare suitable input data for data mining algorithms.

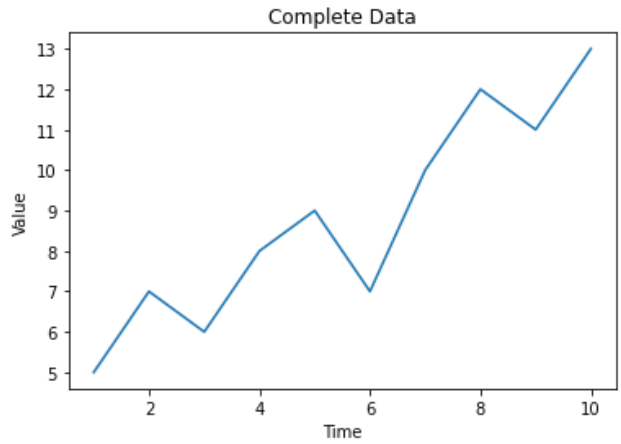
II. Theoretical Summary

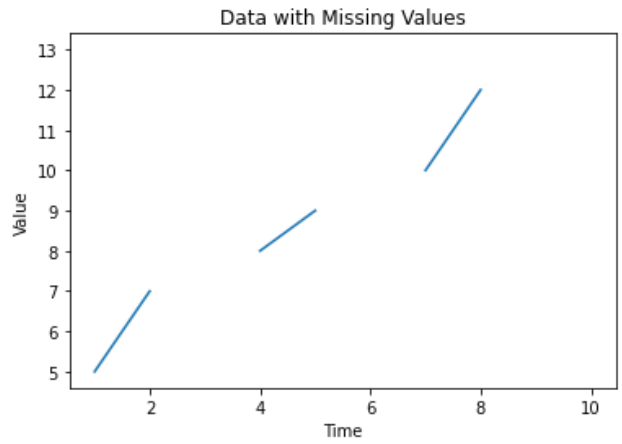
1. Introduction to Data Preprocessing

Data preprocessing is the step of preparing data before applying data mining or machine learning algorithms.

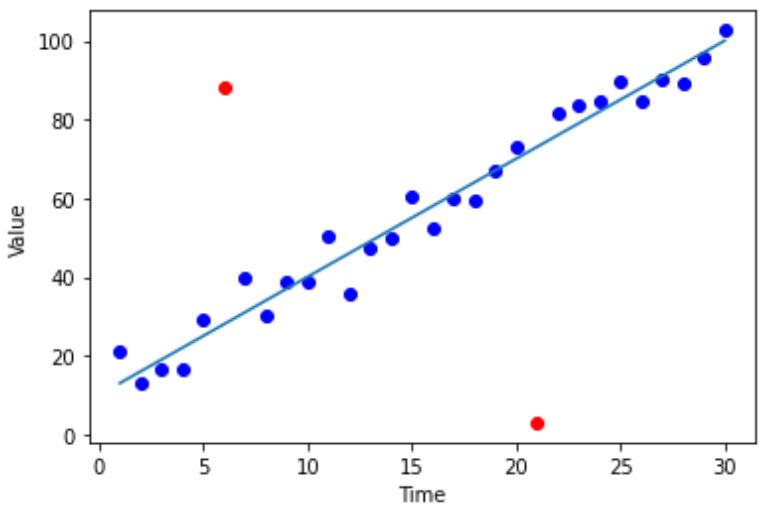
Real-world data is often:

- Missing values

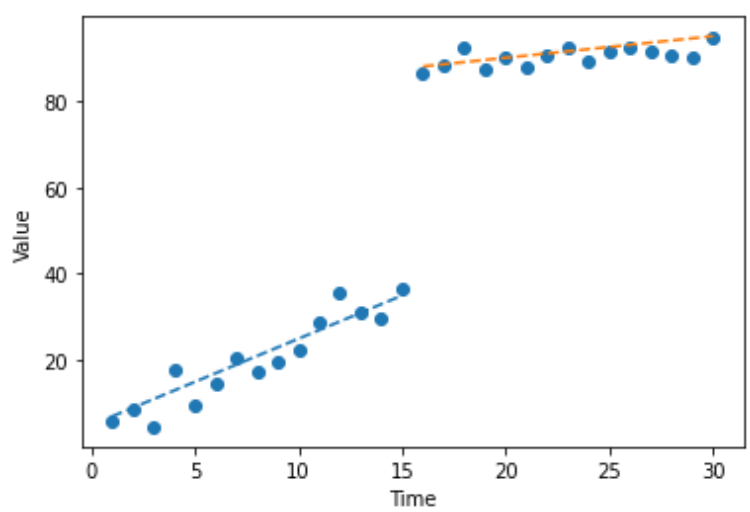




- Noise



- Inconsistent



Objectives:

- Improve data quality
- Reduce data size
- Increase model efficiency and accuracy

2. Data Objects and Types of Attributes

Data objects: Records, samples, or rows of data.

Types of Attributes:

Nominal: No inherent order

Example: gender, color

Ordinal: Has order but no fixed interval between values

Example**:** satisfaction level (low, medium, high)

Interval: Has order and equal intervals, but no absolute zero point

Example: temperature (°C)

Ratio: Has an absolute zero point

Example: age, income, height

3. Data Cleaning

Objective: Handle missing, noisy, or inconsistent data.

Possible methods include:

3.1. Deleting Records:

Example: In a student dataset with 100 rows, 3 students are missing Math scores. If the number of missing values is very small, we can delete those 3 rows to ensure clean data.

Applied when: The missing rate is small and removing the records does not result in significant information loss.

3.2. Replacing with Mean/Median

Example: In an income dataset, one person's income is missing.

We can compute: Average income of the remaining individuals = 15 million VND.
Fill in 15 million VND for the missing value. If the data is skewed (for example, some

individuals have extremely high incomes), we use the median instead of the mean to avoid the influence of outliers.

Applied when:

You want to retain the number of data records

The data distribution is relatively stable

3.3. Interpolation or Prediction

Example: Daily temperature data:

Day 1: 30°C

Day 2: ?

Day 3: 34°C

We can interpolate: Day 2 ≈ 32°C or use a regression model to predict the missing value based on other variables.

Applied when:

The data is continuous over time

There is a clear relationship between variables

3.4. Noise Smoothing:

Using Binning

Example:

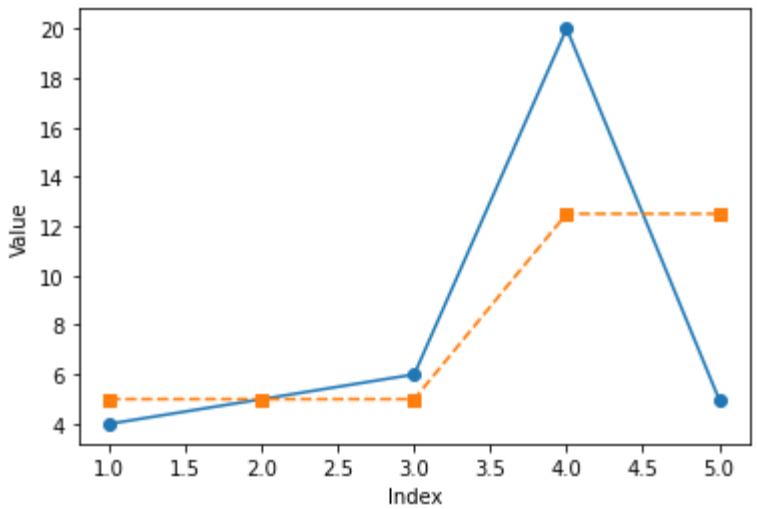
Test scores: 4, 5, 6, 20, 5

Divide into bins of 3 values:

Bin 1: 4, 5, 6 → replace with 5

Bin 2: 20, 5 → replace with 12.5

⇨ Helps reduce the effect of small fluctuations.

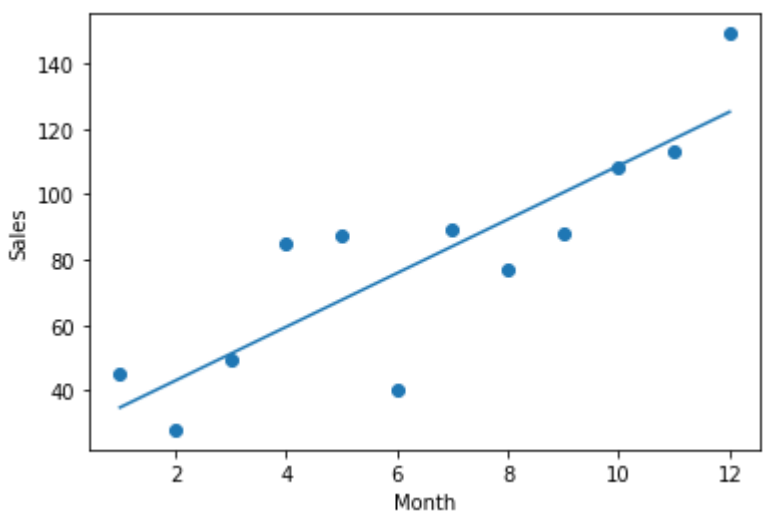


Binning helps smooth data by grouping values into bins.

Using Regression

Example: Monthly sales fluctuate significantly.

We can use linear regression to find the overall trend line instead of focusing on each noisy data point individually.



Regression helps:

- Clearly identify long-term trends
- Reduce the impact of random noise

3.5. Detecting and Removing Outliers

Example: Employee salaries: 10 million, 12 million, 11 million, 9 million, 500 million

The value 500 million is significantly different from the rest.

We can:

- Check whether it is a data entry error
- If it is an input error → remove it
- If it is real data → analyze it separately

Outliers can be detected using:

- Z-score
- IQR (Interquartile Range)
- Boxplot

4. Data Integration

Combining data from multiple sources.

Common issues:

- Duplicate data
- Inconsistencies in units or formats
- Value conflicts

Solutions:

- Remove duplicates
- Standardize formats
- Merge tables using common keys

Example

A sales company collects data from two different sources:

Source 1 (Sales System)

Customer ID	Sales (VND)
A01	5.000.000
A02	7.000.000
A03	6.000.000

Source 2 (Marketing System)

Customer ID	Sales (million VND)
A02	7
A03	6
A04	8

Issues That May Arise:

1. Duplicate data: A02, A03 appear in both tables.
2. Inconsistent units: Table 1: VND, Table 2: million VND
3. Value conflicts: If A02 has different values in the two tables, it must be verified.

Possible solutions:

- Standardize units (convert everything to VND)
- Merge using a common key (Customer ID)
- Remove duplicates

5. Data Reduction

Objective: Reduce data size while preserving important information.

Methods:

- Reduce the number of attributes (Feature Selection)
- Reduce the number of records (Sampling)
- Data compression
- Dimensionality reduction (PCA)

Benefits:

- Faster processing
- Lower storage costs
- Reduced risk of overfitting

6. Data Transformation

Transform data into a suitable format for modeling.

Techniques:

6.1. Normalization

Customer data includes:

- Age: 18 - 60
- Income: 5 million - 200 million

If value ranges differ significantly, the model may become biased toward variables with larger magnitudes.

We can scale the data to the range [0,1]

Formula:

$$x' = \frac{x - min}{max - min}$$

Example: Age 18 - 60 can be transformed into the range 0 - 1

Income 5 - 200 million VND can be transformed into the range 0 - 1

Standardization using mean and standard deviation:

$$x' = \frac{x - \mu}{\sigma}$$

Suitable for models such as:

- Logistic Regression
- SVM
- KNN

6.2. Encoding Categorical Data

Example

Gender variable:

- Male
- Female

Some models or methods cannot process text input, so categorical values must be converted into numbers.

Label Encoding

Male = 0
Female = 1

One-hot Encoding

Male → (1,0)
Female → (0,1)

Suitable when there is no order among the categories.

6.3. Feature engineering

Example: E-commerce data

Suppose we have a dataset table:

Customer ID	Quantity	Product_price	Order date
A01	2	150000	1/5/2024
A02	1	500000	2/10/2024
A03	5	120000	3/15/2024

We can create new features as follows:

1. Total order value

Creating this feature helps the model understand the actual value of an order.

$$Total_amount = Quantity \times Product_price$$

2. Order month

From Order_date, extract: Month, Quarter, Day of the week to analyze seasonal purchasing behavior.

3. Product price group

Classification helps the model learn by groups instead of raw numbers. For example:

- < 200.000 → Low price
- 200.000 – 400.000 → Medium
- 400.000 → High

7. Data Discretization

Convert continuous data into discrete intervals.

Example:

- Age → (0–18), (19–35), (36–60), (>60)

Methods:

- Equal-width binning
- Equal-frequency binning
- Based on decision trees or entropy

III. Environment Setup

1. Using Anaconda

Visit: https://www.anaconda.com/products/distribution

Download the version suitable for your operating system.

2. Using Python + pip

Visit: https://www.python.org/downloads/

Download and install Python (remember to select **Add to PATH**).

3. Using Google Colab or Kaggle

Google Colab

Visit: https://colab.research.google.com

Log in with your Google account.

Select New Notebook.

Kaggle

Visit: https://www.kaggle.com

Log in, then select **Code → New Notebook**.

IV. Practice

Exercise 1. Given the file *bank.csv* with the following attributes:

age: Customer age

job: Occupation

marital: Marital status

education: Education level

default: Has credit in default? (yes/no)

balance: Account balance

housing: Has a housing loan? (yes/no)

loan: Has a personal loan? (yes/no)

contact: Contact communication type

day: Day of the month contacted

month: Month contacted

duration: Call duration (seconds)

campaign: Number of contacts during this campaign

pdays: Number of days since last contact (-1 = never contacted before)

previous: Number of previous contacts

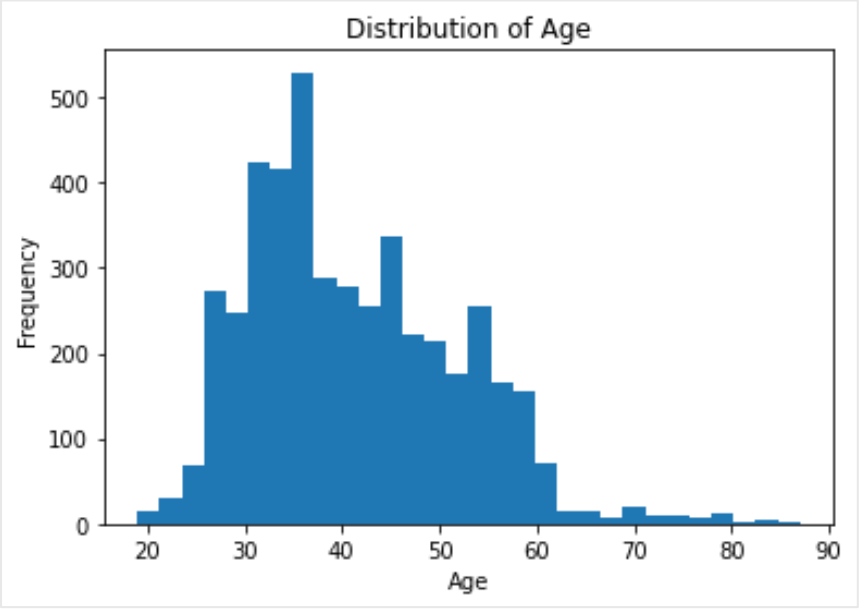
poutcome: Outcome of the previous campaign

y: Whether the customer subscribed to a term deposit (target variable)

Using the file ***bank.csv*** and Python programming language, perform the following tasks:

Data Distribution

1. Draw a histogram for the variable age. At which age range are customers most concentrated ?



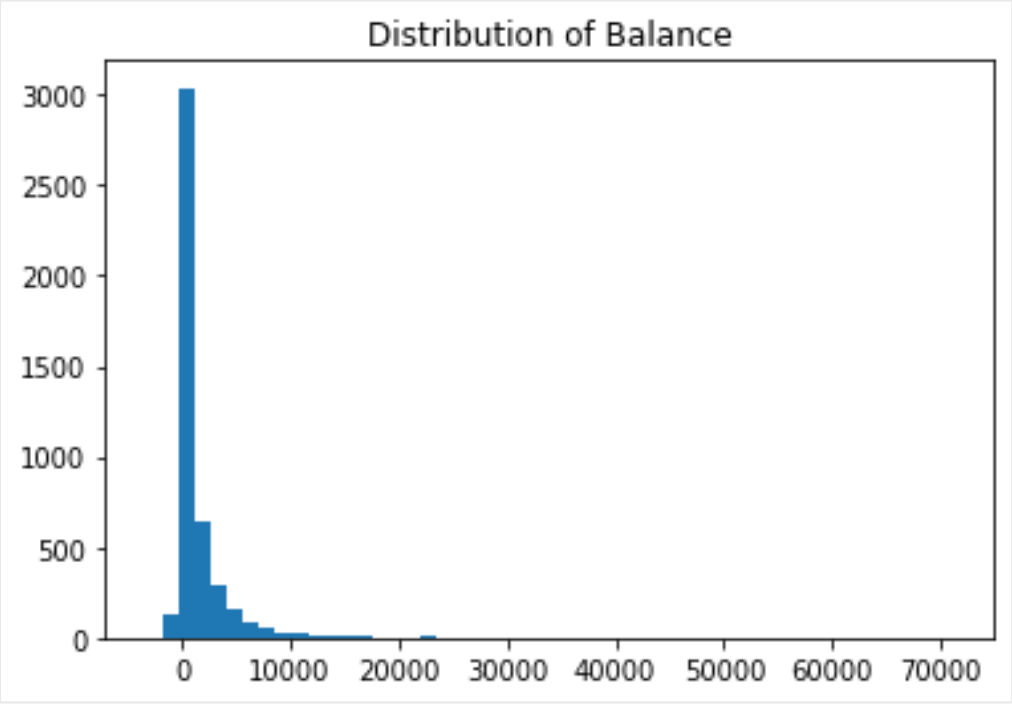
Suggestions: You can use

plt.hist(df['age'], bins=30)

df['age'].plot(kind='hist')

df['age'].describe()

2. Draw a histogram for balance. Are there many outliers ?



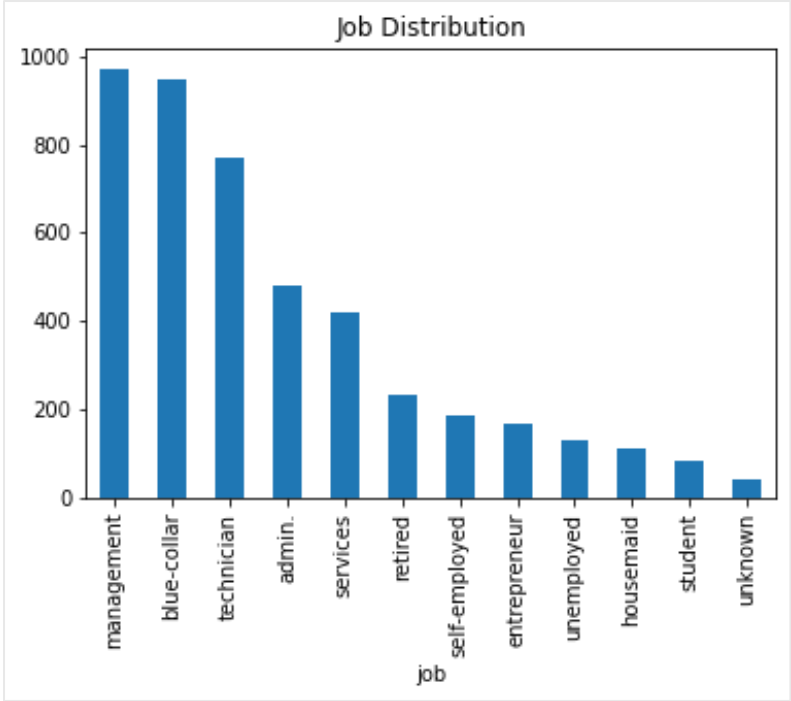
Suggestions: You can use

plt.hist(df['balance'], bins=50)

plt.boxplot(df['balance'])

Dùng IQR: *df['balance'].quantile()*

3. Draw a bar chart for job. Which occupation has the highest proportion?



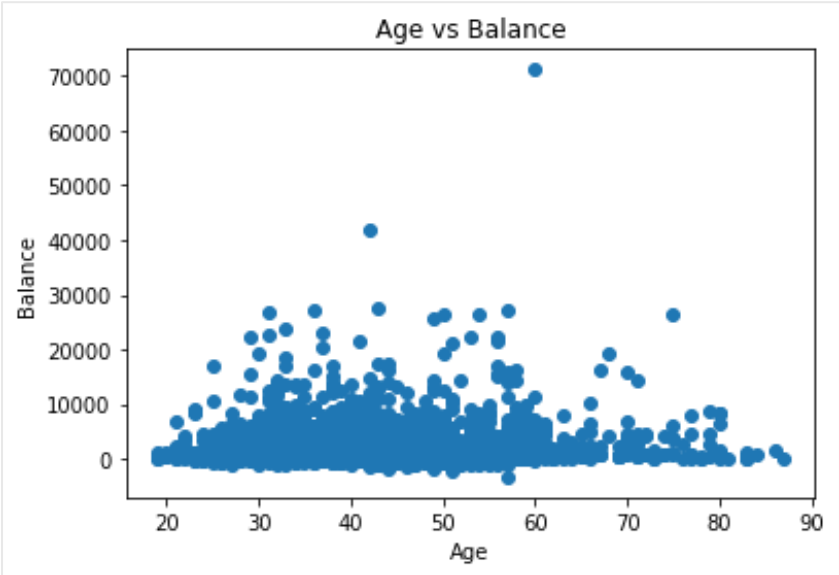
Suggestions: You can use

df['job'].value_counts().plot(kind='bar')

value_counts(normalize=True)

Relationship Between Variables

4. Draw a scatter plot between balance and age. Is there any relationship?

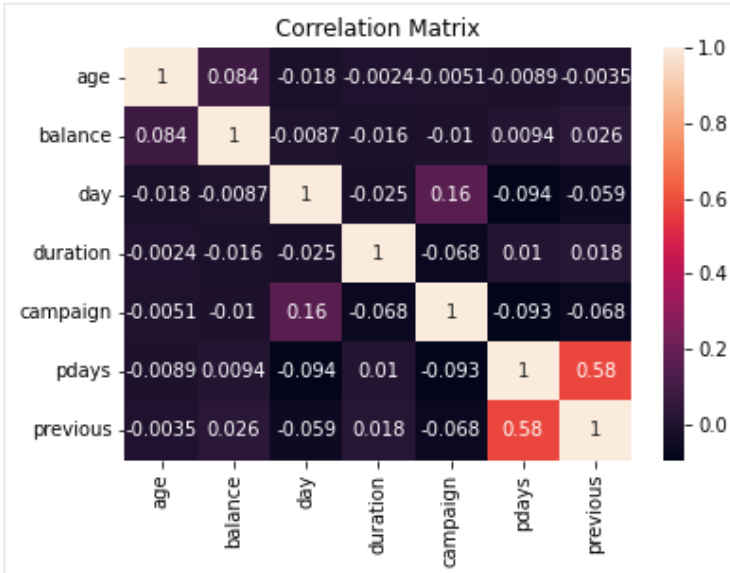


Suggestions: You can use

plt.scatter(df['age'], df['balance'])

Correlation: *df[['age','balance']].corr()*

5. Draw a heatmap of the correlation matrix among numerical attributes.



Suggestions: You can use

df.corr(numeric_only=True)

Draw heatmap: *sns.heatmap()*

6. How does the number of contacts (campaign) affect the outcome y ?

Suggestions: You can use

pd.crosstab(df['campaign'], df['y'], normalize='index')

Boxplot: sns.boxplot(x='y', y='campaign', data=df)

Time-based Analysis

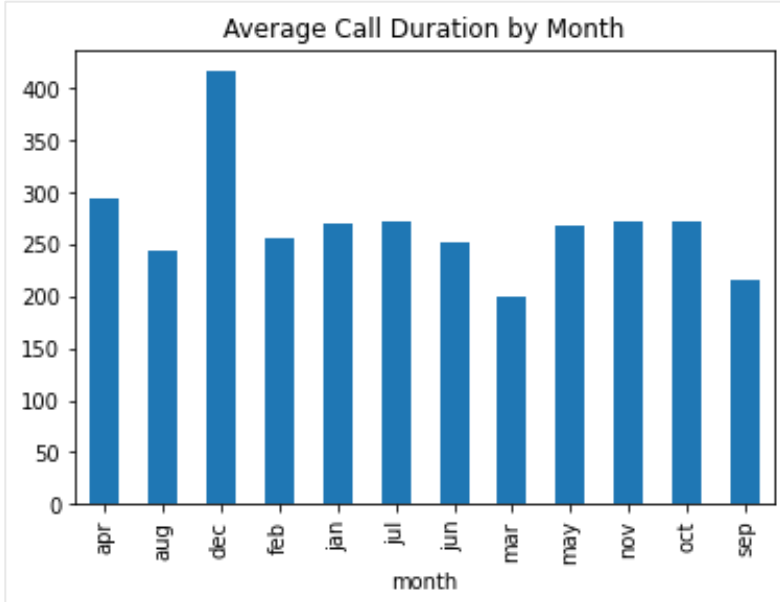
7. Which month has the highest subscription rate?

Suggestions: You can use

pd.crosstab(df['month'], df['y'], normalize='index')

Sort: *.sort_values(by='yes', ascending=False)*

8. Average call duration by month.



Suggestions: You can use

df.groupby('month')['duration'].mean()

Draw: *.plot(kind='bar')*

Data Understanding

9. How many records and how many attributes does the dataset contain ?

Suggestions: You can use

df.shape

df.info()

10. Check for missing values ?

Suggestions: You can use

df.isnull().sum()

Check "unknown": *df.eq('unknown').sum()*

11. What is the target variable ?

12. Are there any abnormal negative values (e.g., balance = -88) ?

Suggestions: You can use

df[df['balance'] < 0]

df.describe()

Data Cleaning

13. Check and handle the value: pdays = -1. What does it mean ? Should it be replaced or converted to NaN ?

Suggestions: You can use

df['pdays'].value_counts()

df['pdays'].replace(-1, np.nan)

14. Handle "unknown" values in columns such as: contact, poutcome

Suggestions: You can use

df.replace('unknown', np.nan)

Replace with mode: *df.fillna(df.mode().iloc[0])*

15. Check for duplicate records and remove them if any.

Suggestions: You can use

df.duplicated().sum()

Delete: *df.drop_duplicates()*

16. Check and handle outliers in: balance, duration

Suggestions: You can use

Boxplot: *sns.boxplot()*

IQR method: quantile(0.25), quantile(0.75)

Data Transformation

17. Normalize numerical variables: age, balance, duration

Suggestions: You can use

from sklearn.preprocessing import StandardScaler

MinMaxScaler()

scaler.fit_transform()

18. Encode categorical variables: job, marital, education, housing, loan Should you use Label Encoding or One-hot Encoding ? Why ?

Suggestions: You can use

One-hot encoding: *pd.get_dummies()*

Label encoding: *LabelEncoder()*

19. Convert the variable month into numeric form (1 - 12).

Suggestions: You can use

Create a dictionary mapping

Use *df['month'].map(dict)*

Feature Engineering

20. Create new variables: Age group (young, middle, senior), Total number of contacts (campaign + previous) and Binary variable: whether previously contacted or not.

Suggestions: You can use

Age grouping: *pd.cut()*

Total contacts: *df['campaign'] + df['previous']*

Binary variable: *np.where()*

21. Create variable: duration_per_campaign = duration / campaign

Suggestions: You can use

df['duration'] / df['campaign']

Imbalanced Data Analysis

22. What is the ratio between y = yes and y = no ?

Suggestions: You can use

df['y'].value_counts(normalize=True)

23. If the data is imbalanced, propose handling methods (undersampling, oversampling, SMOTE).

Suggestions: You can use

Oversampling: *RandomOverSampler()*

Undersampling: *RandomUnderSampler()*

SMOTE: *from imblearn.over_sampling import SMOTE*

Exercise 2. Given the file *titanic.csv* with the following attributes:

pclass: Ticket class (1 = Upper, 2 = Middle, 3 = Lower)

survived: Survival status (1 = survived, 0 = died)

name: Passenger name

sex: Gender (male/female)

age: Age

sibsp: Number of siblings/spouses aboard

parch: Number of parents/children aboard

ticket: Ticket number

fare: Ticket fare

cabin: Cabin number

embarked: Port of embarkation (C =Cherbourg, Q = Queenstown, S = Southampton)

boat: Lifeboat number (if survived)

body: Body identification number (if deceased)

home.dest: Passenger's last known residence

Using the file ***titanic.csv*** and Python, perform the following tasks:

Basic Distribution

1. What is the survival rate ?

Suggestions: You can use

df['survived'].value_counts(normalize=True)

2. Gender distribution (sex)

Suggestions: You can use

df['sex'].value_counts()

3. Ticket class distribution (pclass)

Suggestions: You can use

df['pclass'].value_counts()

4. Age distribution (age)

Suggestions: You can use

plt.hist(df['age'].dropna())

df['age'].describe()

Survival Comparison

5. Survival rate by gender

Suggestions: You can use

pd.crosstab(df['sex'], df['survived'], normalize='index')

6. Survival rate by ticket class (pclass)

Suggestions: You can use

pd.crosstab(df['pclass'], df['survived'], normalize='index')

7. Survival rate by age group

Suggestions: You can use

pd.cut()

Then *crosstab()*

8. Compare fare between survivors and non-survivors using a boxplot

Suggestions: You can use

sns.boxplot(x='survived', y='fare', data=df)

Data Understanding

9. How many rows and columns does the dataset have ?

Suggestions: You can use

df.shape

10. What is the meaning of each variable (pclass, sibsp, parch, boat, body, etc.)?

Suggestions: You can use

df.head()

df.info()

11. Is the survival variable (survived) imbalanced ?

Data Cleaning

12. Check columns with missing values: age, cabin, embarked, boat, body, home.dest

Suggestions: You can use

df['survived'].value_counts(normalize=True)

13. Should we remove columns such as: name, ticket, boat, body ? Why ?

Suggestions: You can use

df.drop(columns=[...])

Consider checking correlation or potential data leakage.

14. Handle missing values in: age (fill with mean/median or by group: sex + pclass ?) embarked (fill with mode ?)

Suggestions: You can use

df.groupby(['sex','pclass'])['age'].transform('median')

Or fillna(df['age'].median())

15. Check and handle duplicate records

Suggestions: You can use

df.groupby(['sex','pclass'])['age'].transform('median')

Hoặc *fillna(df['age'].median())*

Data Transformation

16. Encode categorical variables: sex, embarked, pclass

Suggestions: You can use

pd.get_dummies()

Or *LabelEncoder()*

17. Should we normalize variables such as age and fare? When is normalization necessary?

Suggestions: You can use

Dùng *StandardScaler()*

18. Extract Title (Mr, Mrs, Miss, Master, etc.) from the name column. Should these titles be used as a new feature?

Suggestions: You can use

df['name'].str.extract()

value_counts() để xem nhóm

Feature Engineering

19. Create new features: family_size:

family_size = sibsp + parch + 1

is_alone (whether traveling alone)

Suggestions: You can use

df['sibsp'] + df['parch'] + 1

Create is_alone: *np.where()*

20. Create age groups: child, adult, senior

Suggestions: You can use

pd.cut()

21. Create variable: *cabin_known* (có thông tin cabin hay không)

Suggestions: You can use

df['cabin'].notnull()

np.where()

22. Create variable *fare_per_person*:

fare_per_person = fare / family_size

Suggestions: You can use

df['fare'] / df['family_size']

Outlier Analysis

23. Check for outliers in: fare, age

Suggestions: You can use

sns.boxplot()

Or use the IQR method.

24. Should we handle fare = 0?

Suggestions: You can use

df[df['fare']==0]

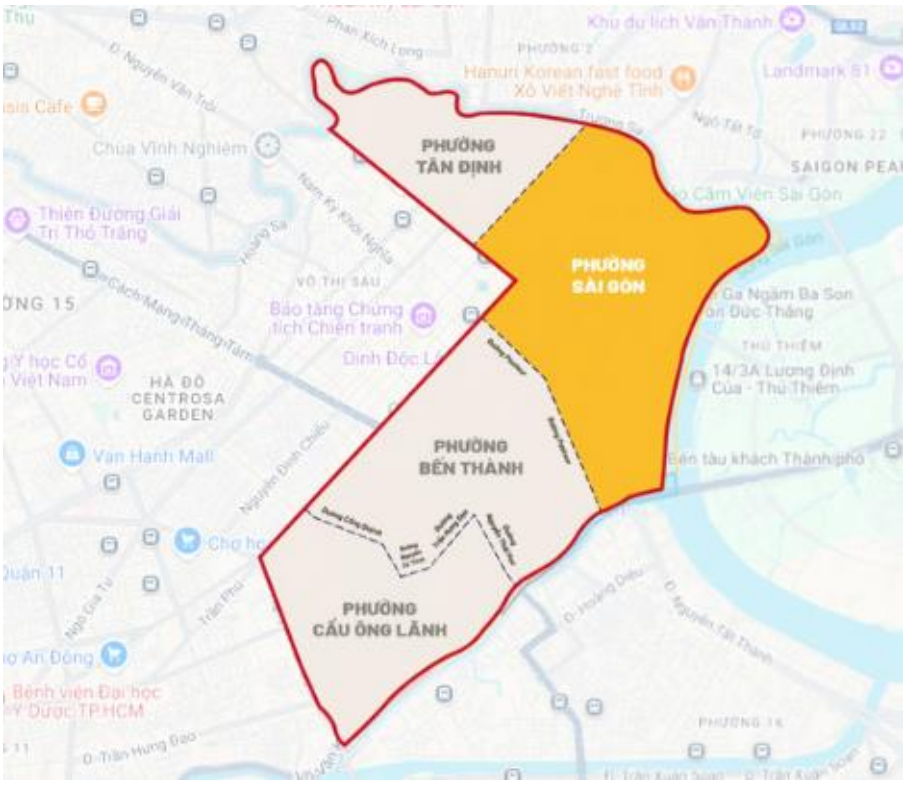
Check its proportion before deciding whether to handle it.

Exercise 3. The file *BDS_TPHCM.csv* contains information related to real estate listings in Ho Chi Minh City. However, the dataset contains many data errors, noise, inconsistencies, and missing values…

Requirement 1: After administrative restructuring, Ho Chi Minh City has changed its addresses and commune/ward divisions. Using the column addressLocality, write code to identify the old addresses of the properties. (You may refer to the column address_old for comparison.)

Example:

For District 1, after restructuring, it was divided into 4 new wards: Tan Dinh Ward, Sai Gon Ward, Ben Thanh Ward, Cau Ong Lanh Ward



Requirement 2: Clean the dataset focusing on the following columns:tính *Chiều dài, Chiều ngang, price_value, area*.

Then calculate **the average price (VND/m²)** for each area based on:

New commune/ward (after restructuring)

Old district (before restructuring)

Old ward – grouped by each old district (before restructuring)

Compare the prices of several nearby areas (within the same old district), analyze whether the pricing is reasonable, and visualize the price differences by area.

You may refer to the following suggestions or implement your own approach:

Outlier Analysis

1. Detect properties with unusually high or low prices.
2. Detect abnormal area values (e.g., 12m² but priced at 192 billion VND).

Data Visualization

3. Draw a histogram of price distribution (**price_value**).

4. Draw a histogram of area distribution (**area**).
5. Compare average price by commune/ward (**addressLocality**).
6. Compare price per m² by property type.
7. Draw a scatter plot between area and price.
8. Draw a boxplot of price by property type.

Data Understanding

9. How many rows and columns does the dataset have?
10. Which columns are numeric?
11. Which columns are categorical?
12. How many listings are there for each property type (Residential land, Villa, Townhouse, etc.)?
13. What is the average property price?

Data Cleaning

14. Check columns with missing values.
15. Standardize the column **price_text** (e.g., "3.3 billion VND", "16 million/m²") into numeric format.
16. Check for duplicate records based on Listing ID.
17. Standardize columns with units such as "m", "m²" (Length, Width, area).